



Polimorfismo e Abstração

Aula 08 · Bloco 2 — Os Pilares da POO

Código que funciona com objetos que ainda não existem

Nesta aula

1. Um comando, **muitos comportamentos**
2. **Upcasting** e ligação dinâmica
3. **Classe abstrata** — um molde que não vira objeto
4. **Interface** — contrato puro
5. Qual usar — e o `instanceof`

Você já fez isso

No desafio da aula 07:

```
Funcionario[] folha = { new Gerente(...), new Vendedor(...), new Estagiario(...) };  
  
for (Funcionario f : folha) {  
    System.out.println(f.getNome() + ": " + f.calcularSalario());  
}
```

Um único laço. Uma única chamada. Três cálculos diferentes.

E nenhum `if` perguntando *"que tipo é esse?"*.

A alternativa sem POO

```
if (tipo.equals("gerente")) {  
    salario = base * 1.2 + AUXILIO;  
} else if (tipo.equals("vendedor")) {  
    salario = base + vendas * 0.05;  
} else if (tipo.equals("estagiario")) {  
    salario = bolsa;  
}
```

Adicionar um cargo aqui significa **caçar todos os** **if** espalhados pelo sistema.

Na versão polimórfica, significa **criar uma classe nova** — e nada do que existe muda.

Upcasting

Uma variável pode ter um tipo **mais genérico** que o objeto:

```
Funcionario f = new Gerente("Ana", "1001", 5000);  
//      ↑ tipo da VARIÁVEL           ↑ tipo do OBJETO
```

É automático e sempre seguro — todo gerente é funcionário.

```
f.calcularSalario();    // ✓ executa a versão do GERENTE  
f.receberBonus();       // ✗ cannot find symbol
```



Duas regras explicam tudo

O **tipo da variável** decide
quais métodos você **pode chamar**.

O **tipo do objeto** decide
qual **versão** do método roda.

A segunda tem nome: **ligação dinâmica**.
O Java só descobre em tempo de execução.

Classe abstrata

`new Funcionario("Ana", 3000)` é possível. E o que essa pessoa faz na empresa?

```
public abstract class Funcionario {  
    protected String nome;  
  
    // MÉTODO ABSTRATO: sem corpo. A subclasse é OBRIGADA a implementar.  
    public abstract double calcularSalario();  
  
    public String getNome() { return nome; }    // concretos convivem  
}  
  
new Funcionario("Ana", 3000);    // ✗ Funcionario is abstract
```

Duas garantias de graça

🚫 Ninguém cria um funcionário **genérico**, sem cargo;

⚖️ Se alguém criar `Diretor extends Funcionario` e **esquecer** o `calcularSalario()`, **não compila**.

O contrato passa a ser cobrado pelo compilador — não pela boa vontade de quem escreve.



Repare nisto

O `toString()` da superclasse
chama `calcularSalario()` —

um método que ela **nem sabe implementar**.

Funciona porque, na execução, quem responde é o
objeto real.

Isso é abstração: programar contra a *ideia* de
funcionário.

Interface: contrato puro

Herança resolve "é um". Mas e capacidades que **atravessam hierarquias**?

Um **Livro** pode ser emprestado. Uma **SalaDeAula** também — e uma não tem nada a ver com a outra.

```
public interface Emprestavel {  
    void emprestar(String responsavel);    // sem corpo  
    void devolver();  
    boolean estaDisponivel();  
}  
  
public class Livro implements Emprestavel { ... }
```

E o código funciona com todas

```
Emprestavel[] itens = { new Livro("Dom Casmurro"), new Projetor("Epson X1") };  
  
for (Emprestavel item : itens) {  
    if (item.estaDisponivel()) item.emprestar("Ana");  
}
```

Livro e projetor não têm parentesco nenhum. Mas assinaram o mesmo contrato.



Uma classe **estende uma** superclasse, mas pode **implementar várias** interfaces.

Interface ou classe abstrata?

	Classe abstrata	Interface
Relação	"é um"	"é capaz de"
Atributos	sim, com estado	só constantes
Métodos concretos	sim	só <code>default</code> / <code>static</code>
Construtor	sim	não
Quantas por classe	uma	várias

Classe abstrata quando as subclasses compartilham dados e comportamento. **Interface** quando você quer só definir uma capacidade.

instanceof e o downcasting

```
if (f instanceof Vendedor v) {           // testa E converte – Java 16+
    System.out.println(v.getTotalVendas());
}

if (f instanceof Vendedor) {             // a forma antiga
    Vendedor v = (Vendedor) f;           // downcasting explícito
}
```

Sem o `instanceof`, um cast errado explode com `ClassCastException`.

instanceof em excesso

é cheiro de polimorfismo mal feito.

Se você escreveu uma cadeia de

```
if (x instanceof A) ... else if (x instanceof B)
```

,

quase sempre o certo era **um método polimórfico** na superclasse.

Use **instanceof** como exceção, não como estratégia.

Exercícios da aula

Na pasta `aula-08/` :

1. **Forma** abstrata + **Circulo** , **Retangulo** , **Triangulo** — soma das áreas **sem** nenhum **if** de tipo;
2. **Funcionario** abstrata — crie um **Diretor** **sem** implementar o método e copie o erro;
3. **Emprestavel** em duas classes sem parentesco + um método que serve às duas;
4. **Pagamento** — Pix, cartão e boleto, cada um com sua taxa;
5. **Desafio** 🌶️ **Reprodutor.java** — **Midia** abstrata + interface **Baixavel** só em algumas.

Fim do Bloco 2

Encapsulamento, herança, polimorfismo, abstração.

Os quatro pilares estão de pé.

Bloco 3 — POO na prática: coleções,
exceções, pacotes e um projeto de verdade.